

UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

POST-MODIFICATION TEST DOCUMENT

Ingegneria del Software: Tecniche Avanzate – Post-Modification Test Document

Mario Celzo	Matricola: NF22500193
Renato Mancino	Matricola: NF22500242
Alfonso Maria Ferrara	Matricola: NF22500210

Repository: `https://github.com/mariocelzo/biblioflow-app`

Anno Accademico 2025-2026

Indice

1 Regression Testing **1**

1.1 Regression Testing per la Change Request 01 (CR-01) 1

1.1.1 Metodologia Operativa 1

1.1.2 Caso 1: Regressione nei Test delle Route di Prenotazione 1

1.1.3 Caso 2: Regressione nel Contratto della Pagina di Prenotazione 2

1.1.4 Analisi di Impatto su Accesso e Area Staff 2

1.2 Regression Testing per la Change Request 02 (CR-02) 3

1.2.1 Strategia di Regressione 3

1.2.2 Esecuzione della Suite alla Chiusura delle Change Requests 3

1.2.3 Analisi sulla Versione Finale 3

1.2.4 Conclusione 4

2 Copertura **5**

2.1 Metodologia 5

2.1.1 Copertura Backend 5

2.1.2 Copertura Frontend 5

2.1.3 Risultati 5

2.2 Analisi 6

3 Unit Testing **8**

3.1 Unit Testing per la Change Request 01 (CR-01) 8

3.1.1 Tecnologie e Framework 8

3.1.2 Logica Core: servizio di dominio delle prenotazioni 8

3.1.3 Modifiche ai test esistenti 9

3.1.4 Funzioni di Autorizzazione 9

3.1.5 Interfaccia: Route di Prenotazione 9

3.2 Unit Testing per la Change Request 02 (CR-02) 9

3.2.1 Stack Tecnologico di Testing 9

3.2.2 Servizi e Automazioni 10

3.2.3 Frontend Testing (React) 10

3.2.4 Middleware della Coda 10

4 Integration Testing **11**

4.1 Integration Testing per la Change Request 01 (CR-01) 11

4.1.1 Strategia di Testing 11

4.1.2 Suite di Test 11

4.2 Integration Testing per la Change Request 02 (CR-02) 12

4.2.1 Strumenti Utilizzati 12

4.2.2 Componenti Sotto Test 12

4.2.3 Suite di Test 12

4.2.4 Risultati 13

5 System Testing **14**

5.1 Aggiornamento Categorie 14

5.1.1 Nuovo Parametro: Titolarità e Concorrenza 14

5.1.2 Nuovo Parametro: Lista d'Attesa (LA, EL) 14

5.2 Nuovi Test Case (TC) 15

6 WebApp Testing (End-to-End) **17**

6.1 Strategia di Testing E2E 17

6.2 Casi di Test E2E Automatizzati 17

Elenco delle tabelle

1.1 Casi di prenotazione modificati dalla CR-01 2
1.2 Casi invariati alla chiusura delle CR, rieseguiti sulla versione finale 4

2.1 Metriche di Copertura del Codice 6
2.2 Difetti emersi dall'aumento della copertura 7

5.1 Nuovi parametri: Titolarità e Concorrenza 14
5.2 Nuovi parametri: Lista d'Attesa 14
5.3 Nuovi Test Case di sistema 16

6.1 Casi di Test E2E 18

CAPITOLO 1

Regression Testing

1.1 Regression Testing per la Change Request 01 (CR-01)

Durante l'implementazione della CR-01 è stata eseguita un'analisi per identificare le regressioni introdotte dalle modifiche alle route di prenotazione e per distinguerle dalle differenze volute.

1.1.1 Metodologia Operativa

Il processo di regression testing è stato condotto seguendo un approccio iterativo per garantire che le nuove funzionalità non compromettessero la stabilità del sistema esistente. Il flusso di lavoro seguito è stato il seguente:

1. **Implementazione delle Modifiche:** sono state applicate le modifiche al codice di produzione necessarie per la CR-01.
2. **Esecuzione della Suite di Test:** è stata rieseguita l'intera suite di caratterizzazione della versione di partenza (34 casi).
3. **Identificazione dei Fallimenti:** l'esecuzione ha evidenziato fallimenti nei casi che descrivevano le route di prenotazione.
4. **Analisi delle Cause:** per ogni fallimento è stato stabilito se derivasse da una regressione o da un cambiamento richiesto dalla CR.
5. **Classificazione:** ogni caso è stato classificato come *invariato*, *modificato* o *eliminato*; i casi modificati conservano il proprio identificativo e indicano il test che ne verifica il nuovo comportamento.
6. **Verifica Finale:** i casi invariati sono stati rieseguiti e confrontati uno per uno con l'esito della versione di partenza.

1.1.2 Caso 1: Regressione nei Test delle Route di Prenotazione

File: `tests/pre-modifica/prenotazioni.test.ts`

Questi casi (TC-PRE-001…010) invocano direttamente i route handler di prenotazione sostituendo Prisma e la sessione con mock costruiti sulla versione di partenza.

Il Problema Dopo la CR tutti e dieci i casi falliscono con lo stesso errore: le route ricavano l'utente con `requireUser()` e traducono gli errori di accesso con la classe `AuthError`, ma il mock storico di `@/lib/auth` esporta soltanto `auth`. Vitest interrompe il caso (`No "AuthError" export is defined on the "@/lib/auth" mock`) prima di qualunque accesso al database e di ogni asserzione.

Soluzione Implementata I mock storici non sono stati riscritti: avrebbero cancellato l'evidenza del comportamento originale, compresi i tre difetti fissati da TC-PRE-004, TC-PRE-007 e TC-PRE-010. I casi sono classificati come **modificati** e ciascuno rimanda alla suite che verifica il comportamento nuovo; per i tre difetti la differenza è voluta.

Caso	Nuovo comportamento	Test che lo verifica
TC-PRE-001	identità dalla sessione, creazione in transazione `Serializable`	`TC-BIB31-001`, auth-prenotazioni
TC-PRE-002, 003	validazione centralizzata prima di ogni scrittura	`TC-BIB27-003…013`
TC-PRE-004	intervallo invertito rifiutato con 422 (prima 201)	`TC-BIB27-005`
TC-PRE-005	conflitto garantito dal vincolo del database, con proposta di coda	`TC-BIB24-DB-001…003`, `TC-BIB31-002`
TC-PRE-006	cancellazione solo del titolare; la promozione della coda (CR-02) è stata collegata alla route nella manutenzione correttiva	auth-prenotazioni, `TC-CODA-DEL-001…005`
TC-PRE-007, 008	check-in con data e orario ricomposti, sessione e titolarità su entrambi i percorsi	finestra di check-in (`TC-FIN-*`), auth-prenotazioni
TC-PRE-009, 010	estensione con le regole centralizzate; durata calcolata sul giorno della prenotazione	`TC-BIB27-006…008`, `015…019`

Tabella 1.1: Casi di prenotazione modificati dalla CR-01

1.1.3 Caso 2: Regressione nel Contratto della Pagina di Prenotazione

File: `tests/pre-modifica/admin-ui.test.ts` (TC-PRE-023, TC-PRE-024)

Il Problema TC-PRE-023 verifica che la pagina `/prenota` invii `userId`, posto, data e orari. Dopo la CR il caso continua a passare: la pagina invia ancora l'identificativo. Il rischio era interpretare questo esito come conferma che il server se ne fidasse ancora. TC-PRE-024, che invia alla route il payload prodotto dalla pagina, fallisce invece per la stessa causa di TC-PRE-001…010: il mock storico di `@/lib/auth` non esporta `AuthError`.

Soluzione Implementata Il caso è classificato come **modificato**: il contratto della pagina è invariato, ma il server ignora l'identificativo ricevuto e usa quello della sessione. Lo dimostra il test di integrazione «ignora userId del payload e crea per l'utente autenticato», che invia l'identificativo di un altro studente e verifica che la prenotazione sia intestata a chi ha la sessione. TC-PRE-024 rimanda alla stessa prova e ai casi di ingresso in coda dal conflitto (`TC-BIB52…54`).

1.1.4 Analisi di Impatto su Accesso e Area Staff

File: `tests/pre-modifica/login.test.ts`, `tests/pre-modifica/admin-ui.test.ts`

I casi di accesso, statistiche, gestione utenti e prestiti (TC-PRE-011…020 e TC-PRE-022) non hanno subito regressioni. Le nuove funzioni di autorizzazione sono state aggiunte accanto

a quelle esistenti, senza cambiarne il comportamento: il provider di accesso e le guardie di ruolo dell'area staff sono rimasti compatibili con i casi storici.

1.2 Regression Testing per la Change Request 02 (CR-02)

La CR-02 aggiunge un'entità e nuovi percorsi (coda, promozione, scadenza), ma interviene anche su punti esistenti: l'annullamento dello staff, il rilascio automatico dei no-show e il canale in tempo reale. Il regression testing ha verificato che queste estensioni non alterassero i comportamenti preesistenti.

1.2.1 Strategia di Regressione

La strategia si è concentrata su due momenti:

1. **Chiusura delle Change Requests:** riesecuzione dei 20 casi classificati come invariati, con confronto caso per caso con l'esito della versione di partenza.
2. **Versione finale:** riesecuzione degli stessi casi dopo la manutenzione correttiva successiva alle Change Requests, per distinguere le regressioni dalle correzioni intenzionali.

1.2.2 Esecuzione della Suite alla Chiusura delle Change Requests

I 20 casi invariati sono stati rieseguiti sulla versione che chiude l'implementazione (commit `adc5e30`), eseguiti il 4 settembre 2026, e confrontati con l'esecuzione della versione di partenza.

Risultati:

- **Casi invariati:** 20 passati su 20, identici alla versione di partenza.
- **Aree verificate:** accesso (TC-PRE-011…013), protezione di pagine e API (TC-PRE-014…016), area staff (TC-PRE-017…020, TC-PRE-022), automazioni e notifiche preesistenti (PRE-AUT-001, 002, 004, PRE-NOT-001, 002), contratto del canale SSE (PRE-SSE-001…004).

Due casi modificati continuano a passare dopo la CR, per motivi diversi. PRE-AUT-003 (no-show automatico) gira su database reale: la coda è vuota e la promozione termina con esito `coda_vuota`, senza effetti visibili dal caso storico. TC-PRE-021 (annullamento dello staff) passa perché la promozione, effetto collaterale *best-effort* della route, fallisce sui dati del mock storico (prenotazione senza orari, client senza `$transaction`) e l'errore viene intercettato senza alterare la risposta: il caso non verifica la promozione, verificata invece da `TC-BIB49-001...005` e `TC-BIB58-001, 002`.

1.2.3 Analisi sulla Versione Finale

Dopo le Change Requests il progetto ha attraversato una fase di manutenzione correttiva. La riesecuzione degli stessi 20 casi sulla versione finale dà il seguente quadro:

Casi	Classificazione	Motivo
TC-PRE-011…015, 017…019, 022, PRE-AUT-004, PRE-NOT-001, 002	Invariato, **PASS**	–
TC-PRE-016	Modificato	Il middleware ora verifica davvero il token: un cookie inventato non apre più `/prenota` (si veda il Test Incident n.4).
TC-PRE-020	Modificato	La restituzione registrata dallo staff ora incrementa anche le copie disponibili del libro in una transazione; prima ogni restituzione al banco «perdeva» una copia. Il mock storico non prevede la transazione; il comportamento nuovo è verificato su database reale.
PRE-AUT-001, 002	Modificato	Gli orari delle prenotazioni sono ora salvati nell'ora di Roma, mentre la fixture storica li scrive in UTC: il promemoria non scatta più all'orario della fixture. Il comportamento è verificato da `tests/unit/automation-service.test.ts`.
PRE-SSE-001…004	Eliminato	La catena SSE non è mai stata collegata all'interfaccia e, tenendo i client in memoria, non funziona con più istanze serverless. È stata rimossa e sostituita dalla rilettura autenticata delle API e dal controllo periodico delle notifiche.

Tabella 1.2: Casi invariati alla chiusura delle CR, rieseguiti sulla versione finale

1.2.4 Conclusione

Il Regression Testing ha dato esito positivo in entrambi i momenti. Alla chiusura delle Change Requests tutti i 20 casi invariati conservano l'esito della versione di partenza. Sulla versione finale restano 12 casi invariati, tutti superati; gli altri 8 sono stati modificati o eliminati da interventi documentati, ciascuno con un test che ne verifica il nuovo comportamento.

CAPITOLO 2

Copertura

In questo capitolo vengono presentati i risultati della copertura del codice (code coverage) per il codice server e per l'interfaccia di BiblioFlow.

2.1 Metodologia

Per garantire una valutazione completa dell'efficacia della suite di test, sono state misurate sia la Statement Coverage (copertura delle istruzioni) sia la Branch Coverage (copertura dei rami).

2.1.1 Copertura Backend

La copertura del codice server (route handler, librerie, middleware, hook) è stata calcolata con Vitest e il fornitore di copertura nativo di V8, che misura anche i rami per individuare i percorsi sfuggiti a una semplice verifica delle righe.

- **Strumento:** Vitest 4.1.11 con `@vitest/coverage-v8`.
- **Comando:**

```
npm run test:coverage
```

- **Configurazione:** la misura include tutto `src`, route handler compresi, ed esclude la suite di caratterizzazione. Sono configurate soglie minime: 90% di istruzioni e 80% di rami su `src/lib` e `src/app/api`, 100% sul middleware; sotto soglia il comando fallisce.

2.1.2 Copertura Frontend

I componenti React toccati dalle Change Requests sono verificati in un DOM simulato (jsdom) con Testing Library, nella stessa esecuzione di Vitest.

- **Strumento:** Vitest, jsdom 26, Testing Library.
- **Configurazione:** test dei componenti in `tests/unit/componenti`. Le pagine Next.js sono esercitate dalla suite E2E su build di produzione, che non contribuisce alla misura V8.

2.1.3 Risultati

La tabella seguente riassume le metriche di copertura ottenute, confrontate con quelle misurate all'inizio dell'attività di miglioramento della suite.

Componente	Statement	Branch	Prima (stmt / branch)
Route handler (`src/app/api`, 37 route)	**95,1%**	**89,0%**	72,3% / 66,6%
Librerie (`src/lib`, 29 moduli)	**95,4%**	**90,5%**	89,9% / 83,6%
Middleware	100%	100%	100% / 100%
Hook di ingresso in coda	94,1%	84,4%	25,5% / 25,0%
Totale codice server	**95,2%**	**89,4%**	–
Moduli toccati dalle CR (18 file, UI compresa)	87,4%	80,3%	–
Componenti (`src/components`)	21,4%	17,9%	circa 3%

Tabella 2.1: Metriche di Copertura del Codice

La suite conta 123 file e 1013 test, tutti superati: 815 test di unità, 32 test di componenti, 138 test di integrazione e 28 test post-modifica su database reale (7 sui vincoli del database, 21 su sicurezza e validazione delle API). Prima dell'attività i test erano 763 in 103 file.

2.2 Analisi

Codice server: il codice server ha raggiunto un grado di copertura elevato (95,2% delle istruzioni, 89,4% dei rami), sopra le soglie del Master Test Plan. È importante notare che la configurazione iniziale escludeva dalla misura proprio i route handler: il dato pubblicato in precedenza non comprendeva le API. Misurate per la prima volta, le route si fermavano al 72,3% delle istruzioni, con sette route mai eseguite da alcun test (registrazione, verifica e reinvio dell'email, profilo, dettaglio di un posto e di un libro, storico di un utente).

Di conseguenza, sono stati introdotti test mirati per colmare questo divario:

1. **Route a copertura zero o bassa:** test per registrazione e verifica dell'email su database reale, per dettaglio posti e utenti, anomalie, prestiti dello staff, catalogo, sale, profilo e notifiche, inclusi accessi senza sessione e con ruolo errato.
2. **Errori interni:** un test trasversale chiama 49 handler con il database guasto e verifica che rispondano 500 con un messaggio generico, senza nomi di tabelle, driver o stack.
3. **Ciclo di vita della coda e modifica dello staff** su database reale, componenti dell'interfaccia in jsdom e una suite E2E.

I nuovi test hanno fatto emergere quindici difetti, tutti corretti. Tredici sono coperti da test automatici (per D-02 e D-10 è stato verificato che il test fallisce sul codice precedente); D-14 e D-15 riguardano pagine Next.js senza test unitari e sono stati verificati con la revisione del codice e, indirettamente, dalla suite E2E:

ID	Difetto	Origine
D-01	Registrazione ed estensione registrate nel log come «prenotazione creata».	Baseline; estensione dalla CR-01
D-02	Nel dettaglio di un posto, «oggi» calcolato nel fuso del server: tra mezzanotte e le due di Roma la prenotazione in corso spariva.	Baseline

ID	Difetto	Origine
D-03	I no-show rilevati dal cron non comparivano tra le anomalie né nei contatori dello staff.	Baseline
D-04	«Risolvi tutte» chiudeva un solo evento per utente.	Baseline
D-05	Solleciti e rinnovi registrati come restituzioni e nuovi prestiti; il sollecito singolo non verificava lo stato del prestito.	Baseline
D-06	Durata del prestito decisa dal client, anche negativa o di anni.	Baseline
D-07…09	Valori fuori dominio (filtro sullo stato del prestito, tipo della notifica creata dallo staff, filtri di data delle prenotazioni) producevano un errore 500 invece di 422.	Baseline
D-10	Una modifica dello staff in sovrapposizione, rifiutata dal vincolo del database, rispondeva «errore interno» invece di 409.	Effetto a catena della CR-01
D-11	«1 disponibili», «1 risultati disponibili».	Baseline
D-12	Pulsanti dello staff fatti solo di icona senza nome accessibile; azioni proposte su prenotazioni già concluse.	Baseline e manutenzione correttiva
D-13	Nessuna schermata mostrava allo studente le proprie richieste in coda né permetteva di uscirne.	CR-02 (interfaccia incompleta)
D-14	Le prenotazioni scadute per promozione non confermata non comparivano nello storico.	CR-02
D-15	Una data non valida nell'indirizzo della pagina staff delle prenotazioni causava un errore del server.	Baseline

Tabella 2.2: Difetti emersi dall'aumento della copertura

Interfaccia: la copertura dei componenti passa da circa il 3% al 21,4%. Il valore riguarda tutti i componenti dell'applicazione: quelli toccati dalle Change Requests (mappa, griglia mobile, lista d'attesa, azioni dello staff) sono verificati in dettaglio, mentre le pagine sono esercitate dalla suite E2E. Per questo motivo il totale grezzo dell'intero `src` (48,5%) non è indicativo e non viene usato come soglia.

CAPITOLO 3

Unit Testing

3.1 Unit Testing per la Change Request 01 (CR-01)

Questa sezione descrive i test di unità che verificano la CR-01: prenotazione sicura e atomica, compresi quelli aggiunti nella manutenzione correttiva (`TC-FIN`, `TC-TOL`, `TC-MP`) e nell'aumento della copertura. L'obiettivo era verificare in isolamento le regole del servizio di dominio, le funzioni di autorizzazione e il comportamento delle route.

3.1.1 Tecnologie e Framework

Per lo sviluppo della suite di test sono stati impiegati i seguenti strumenti:

- **Vitest:** framework di testing primario, per la definizione dei casi, le asserzioni e la misura della copertura.
- **Mock di Vitest** (`vi.mock`, `vi.hoisted`): sostituiscono il client Prisma e la sessione di Auth.js, isolando la logica dal database e dall'autenticazione.
- **Richieste HTTP native:** le route vengono invocate con oggetti `Request` costruiti nel test, esattamente come le chiamerebbe Next.js.

3.1.2 Logica Core: servizio di dominio delle prenotazioni

File: `tests/unit/prenotazioni-service.test.ts` (42 casi)

- `TC-BIB27-001...029` (più `010b`): validazione centralizzata degli intervalli, tra cui fine non successiva all'inizio (`005`), durata minima di un'ora e massima di otto (`006...008`), orari della sala e sala inattiva (`012`, `013`), conflitti sul posto e sull'utente con proposta di coda (`015...019`), orari già passati, giorni di chiusura e limite di trenta giorni (`021...029`).
- `TC-BIB31-001`: la creazione avviene in una transazione `Serializable`.
- `TC-BIB31-002`: la violazione del vincolo del database viene tradotta in un conflitto proponibile come coda, non in un errore interno.
- `TC-MP-001...003`: il margine pendolare è letto dal database, non dal corpo della richiesta.
- `TC-BIB31-003...009`: operazioni di dominio della coda e della promozione, usate dalla CR-02.

3.1.3 Modifiche ai test esistenti

- TC-PRE-001…010 non sono stati riscritti: restano come evidenza della versione di partenza (Capitolo 1).
- TC-PRE-016 è stato capovolto e reso asincrono quando il middleware ha iniziato a verificare il token (Test Incident n.4).
- `TC-EST-002` verifica ora anche il tipo di evento `PRENOTAZIONE_ESTESA` dell'estensione (difetto D-01).

3.1.4 Funzioni di Autorizzazione

File: `tests/unit/auth-helpers.test.ts`

- `TC-BIB26-001...007`: `requireUser` e `requireRole` restituiscono 401 senza sessione e 403 per il ruolo non ammesso; `assertOwnership` ammette il proprietario, risponde 404 allo studente che chiede la risorsa di un altro utente e 403 allo staff (`007`).

3.1.5 Interfaccia: Route di Prenotazione

File: `tests/unit/prenotazioni-elenco-e-creazione.test.ts`, `tests/unit/prenotazioni-regole-finestra-checkin.test.ts`, `tests/unit/errori-generici-route.test.ts`

- L'elenco è sempre ristretto alla sessione, anche se l'indirizzo contiene l'identificativo di un altro utente; filtri di data non validi producono 422.
- La creazione ignora l'identificativo inviato dal client e, in caso di conflitto, risponde 409 con la proposta di entrare in coda.
- La finestra di check-in è verificata ai suoi estremi: quindici minuti prima e dopo l'inizio, trenta per chi ha il margine pendolare (`TC-FIN-*`, `TC-TOL-*`).
- Con il database guasto ogni route risponde 500 senza dettagli interni.

3.2 Unit Testing per la Change Request 02 (CR-02)

La CR-02 introduce la lista d'attesa e la promozione automatica. La strategia di testing unitario ha coperto tutti i livelli coinvolti: servizi, automazioni, route e interfaccia.

3.2.1 Stack Tecnologico di Testing

Data la natura dell'applicazione, sono stati adottati strumenti specifici per ogni livello:

- **Vitest con mock:** per i servizi, le automazioni e le route, isolando database, orologio e sessione.
- **Testing Library, user-event e jsdom:** per i componenti React. Permettono di verificare il rendering, lo stato e le interazioni in un DOM simulato, interrogando gli elementi per ruolo e nome accessibile come farebbe una tecnologia assistiva.

3.2.2 Servizi e Automazioni

File: `tests/unit/automation-service.test.ts`, `tests/unit/cron-automations.test.ts`, `tests/unit/prenotazioni-cancellazione-promuove-coda.test.ts`, `tests/unit/admin-prenotazioni.test.ts`

- `TC-BIB40`, `TC-BIB42`, `TC-BIB44` e `TC-BIB46`: il no-show avvia la promozione; le notifiche di ingresso in coda, promozione e scadenza (l'uscita dalla coda registra solo l'evento `CODA_ANNULLATA`); la scadenza delle promozioni non confermate dopo quindici minuti; la correlazione tra notifiche ed eventi.
- `TC-BIB41-001...003`: il lock impedisce due esecuzioni contemporanee del cron.
- `TC-CODA-DEL-001...005` e `TC-CODA-PATCH-*`: la cancellazione dello studente promuove il primo in coda per lo stesso intervallo, solo dopo che la prenotazione risulta annullata, e notifica il promosso.
- `TC-BIB49-001...005`: l'annullamento dello staff promuove e restituisce il nome del promosso all'interfaccia.

3.2.3 Frontend Testing (React)

File: `tests/unit/componenti/*.test.tsx`, `tests/unit/mappa-coda.test.ts`, `tests/unit/mobile-coda.test.ts`, `tests/unit/prenota-conflitto.test.ts`

- **Mappa dei posti:** legenda, posti interattivi, uso da tastiera (Invio e spazio), ingresso in coda con lo stesso contratto della vista mobile, ripiego sulla griglia sotto 768 pixel.
- **Griglia mobile:** ordine naturale dei posti, nome accessibile e stato premuto, posti in manutenzione non selezionabili, ingresso in coda, rifiuto del server, assenza di rete.
- **Lista d'attesa dello studente:** elenco con posizione, uscita dalla coda con rilettura delle posizioni, stato vuoto, errore con «Riprova» (`TC-UI-CODA-001...005`).
- **Azioni dello staff:** azioni proposte solo per gli stati accettati dal server, esito della promozione dopo l'annullamento, conflitto 409 mostrato all'operatore.
- **Conflitto all'ultimo istante:** il messaggio di posto non più disponibile propone la lista d'attesa (`TC-BIB54-001...004`).

3.2.4 Middleware della Coda

File: `tests/unit/middleware-coda.test.ts`

- `TC-BIB51-001...005`: la route della coda è dichiarata nel matcher e respinge le richieste senza sessione (401) o con un cookie inventato; un token firmato davvero viene accettato e la protezione delle API dello staff resta invariata.

CAPITOLO 4

Integration Testing

4.1 Integration Testing per la Change Request 01 (CR-01)

Questa sezione documenta le attività di integration testing svolte per validare la CR-01. Mentre lo unit testing isola i singoli componenti, l'integration testing verifica che route, servizio di dominio e database collaborino correttamente e producano l'esito atteso in uno scenario reale.

4.1.1 Strategia di Testing

Per i test di integrazione è stata adottata una strategia black-box parziale:

- **Ambiente Reale:** i test usano PostgreSQL 16 reale, con le migrazioni applicate e un database isolato il cui nome deve dichiarare di essere di test; i dati vengono ripuliti prima di ogni caso.
- **Esecuzione dal Route Handler:** le richieste attraversano la route, il servizio di dominio e il database, senza mock di Prisma. Fanno eccezione le suite di autenticazione e autorizzazione (`auth-prenotazioni.test.ts`, `autorizzazione-admin.test.ts`), che sostituiscono Prisma e la sessione con mock perché verificano le guardie, che rispondono prima di ogni accesso ai dati.
- **Validazione dell'Esito:** si verificano la risposta HTTP e lo stato finale del database (prenotazioni, eventi, notifiche).

4.1.2 Suite di Test

Concorrenza (`tests/integration/concorrenza.test.ts`)

- **Due richieste simultanee, tre round:** sullo stesso posto e intervallo esattamente una richiesta ottiene la prenotazione.
- **N richieste simultanee:** un solo vincitore e nessun record fantasma lasciato dalle transazioni rifiutate.

Vincolo del database (`tests/post-modifica/prenotazioni-constraint-db.test.ts`)

- `TC-BIB24-DB-001...003`: il vincolo di esclusione, verificato scrivendo direttamente sul database, considera occupato lo slot anche durante il check-in, ammette intervalli adiacenti e ignora le prenotazioni in stato terminale.

Autenticazione e titolarità (`tests/integration/auth-prenotazioni.test.ts`, `tests/integration/autorizzazione-admin.test.ts`)

- 401 su ogni operazione protetta senza sessione; 404 su lettura, modifica e cancellazione della prenotazione di un altro studente.
- Identificativo del payload ignorato: la prenotazione è intestata a chi ha la sessione.
- `TC-BIB57-*` (36 casi): 401 senza sessione e 403 per il ruolo non ammesso sulle sei route dello staff più esposte (prenotazioni, statistiche, anomalie, utenti, posti, richieste), comprese le tre rimaste aperte alla prima verifica; 401 sulla coda (`040`…`042`) e casi in cui la guardia ammette lo staff.

4.2 Integration Testing per la Change Request 02 (CR-02)

L'integration testing per la CR-02 ha l'obiettivo di verificare la catena che parte da un evento di liberazione del posto e arriva alla promozione, alle notifiche e agli eventi registrati.

4.2.1 Strumenti Utilizzati

- **Vitest:** runner dell'intera suite.
- **PostgreSQL 16:** database reale effimero, in locale e nel job `test` della pipeline.
- **Prisma Client:** usato dai test per preparare i dati e verificare lo stato finale.

4.2.2 Componenti Sotto Test

I test di integrazione coinvolgono la cooperazione tra:

- **Route handler:** annullamento dello staff, coda, cron (la cancellazione dello studente è verificata con mock da `TC-CODA-DEL-*` e su database reale dallo scenario E2E-11).
- **Servizio di dominio:** ingresso, uscita, posizione e promozione.
- **Servizio delle automazioni:** no-show, scadenza delle promozioni, lock.
- **Database:** vincolo di unicità delle richieste attive e ordinamento della coda.

4.2.3 Suite di Test

Test 1: Ciclo coda–promozione (`tests/integration/coda-promozione.test.ts`)

1. B entra in coda per il posto prenotato da A.
2. Lo staff annulla la prenotazione di A.
3. B viene promosso con prenotazione, notifica ed evento correlati (`TC-BIB58-001`); con la coda vuota nessuno viene promosso (`TC-BIB58-002`).

Test 2: Automazioni (`tests/integration/automazioni.test.ts`)

1. Il no-show automatico promuove il primo in coda (`TC-BIB47-001`).
2. Due esecuzioni ravvicinate non producono promozioni, notifiche o richieste duplicate (`002`, `002b`); due invocazioni concorrenti del cron eseguono una sola volta (`003`).

3. La promozione non confermata scade e il posto passa al successivo (`005`); la scadenza è idempotente (`006`) e non tocca una promozione confermata con il check-in (`007`).

Test 3: Ciclo di vita della coda (`tests/integration/coda-ciclo-di-vita-db.test.ts`, `tests/post-modifica/lista-attesa-db.test.ts`)

1. Tre utenti ottengono le posizioni 1, 2 e 3 in ordine di arrivo; non si entra due volte nella stessa coda.
2. Il titolare esce dalla coda: resta traccia nel log e chi era dietro avanza; la richiesta di un altro utente risulta inesistente.
3. Dopo l'uscita si può rientrare, in fondo; con lo slot ancora occupato nessuno viene promosso.

4.2.4 Risultati

Tutti i 166 test di integrazione e post-modifica sono stati eseguiti con successo, confermando che promozione, notifiche ed eventi restano coerenti anche con esecuzioni ripetute e concorrenti.

CAPITOLO 5

System Testing

A seguito dell'implementazione delle Change Requests CR-01 (prenotazione sicura e atomica) e CR-02 (lista d'attesa e riassegnazione), la strategia di System Testing è stata aggiornata. Oltre alla riesecuzione dei casi esistenti descritti nel documento «Pre-Modification Test Document», sono stati introdotti nuovi casi di test per coprire le nuove funzionalità.

5.1 Aggiornamento Categorie

Le categorie sono state estese per includere i parametri rilevanti per la titolarità, la concorrenza e la lista d'attesa. Oltre alle categorie già definite nel documento pre-modifica, sono state aggiunte le seguenti.

5.1.1 Nuovo Parametro: Titolarità e Concorrenza

Categoria	Codice	Scelte (Classi di Equivalenza)
Titolarità (TI) Proprietario della risorsa richiesta.	TI0	Propria
	TI1	Di un altro utente
Concorrenza (CC) Richieste contemporanee sullo stesso posto e intervallo.	CC0	Richiesta singola
	CC1	Richieste simultanee

Tabella 5.1: Nuovi parametri: Titolarità e Concorrenza

5.1.2 Nuovo Parametro: Lista d'Attesa (LA, EL)

Categoria	Codice	Scelte (Classi di Equivalenza)
Stato della Coda (LA) Richieste in attesa per il posto e l'intervallo.	LA0	Coda vuota
	LA1	Altri utenti in coda
	LA2	Richiesta attiva dello stesso utente
Evento di Liberazione (EL) Causa per cui il posto torna libero.	EL0	Nessuno (slot ancora occupato)
	EL1	Cancellazione dello studente
	EL2	Annullamento dello staff
	EL3	No-show rilevato dal cron
	EL4	Promozione non confermata scaduta

Tabella 5.2: Nuovi parametri: Lista d'Attesa

5.2 Nuovi Test Case (TC)

In base alle nuove categorie identificate, sono stati progettati e implementati i seguenti Test Case aggiuntivi. Ogni caso è automatizzato: la colonna dell'oracolo indica il test che lo esegue.

TC ID	Test Frame (Parametri)	Oracolo / Risultato Atteso
TC-POST-001	SE0, RS1, OP0…OP4	**Senza sessione:** 401 su ogni operazione sulle prenotazioni e sulla coda (auth-prenotazioni, `TC-BIB57-040…042`).
TC-POST-002	SE1, RU0, TI1, OP1	**Risorsa altrui:** 404 su lettura, `PATCH` di cancellazione e `DELETE`; nessuna scrittura (auth-prenotazioni).
TC-POST-003	SE1, RU0, OP0, OR0, TI1	**Identità nel payload:** la prenotazione è intestata a chi ha la sessione, non all'utente indicato nel corpo (auth-prenotazioni).
TC-POST-004	SE1, RU0, RS1, FS0, FS1, FS3	**Studente sull'area staff:** 403 sulle route dello staff verificate (`TC-BIB57-010…015`, `050`, `051`, `054`, `055`).
TC-POST-005	SE1, RU0, OP0, IO2	**Intervallo invertito:** 422, nessuna prenotazione (`TC-BIB27-005`; prima TC-PRE-004 rispondeva 201).
TC-POST-006	SE1, RU0, OP0, IO0, DI0, CC1	**Richieste simultanee:** una sola prenotazione creata, le altre 409, nessun record fantasma (concorrenza).
TC-POST-007	SE1, RU0, OP0, IO0, DI1, CC0	**Posto occupato:** 409 con la proposta di entrare in coda (`TC-BIB31-002`, `TC-BIB27-015`).
TC-POST-008	SE1, RU0, LA0, LA1	**Ingresso in coda:** posizioni 1, 2, 3 in ordine di arrivo (`TC-CODA-VITA-001`).
TC-POST-009	SE1, RU0, LA2	**Richiesta duplicata:** rifiutata; resta una sola richiesta attiva (`TC-CODA-VITA-002`, `TC-BIB23-DB-001`).
TC-POST-010	SE1, RU0, TI0, LA1	**Uscita dalla coda:** richiesta `ANNULLATA`, evento registrato, chi era dietro avanza (`TC-CODA-VITA-010`).
TC-POST-011	EL1, LA1	**Cancellazione dello studente:** il primo in coda ottiene una prenotazione `CONFERMATA` e la notifica di promozione (`TC-CODA-DEL-001…003`, E2E-11).
TC-POST-012	SE1, RU1, EL2, LA1	**Annullamento dello staff:** promozione del primo in coda e nome del promosso restituito allo staff (`TC-BIB58-001`, `TC-BIB49-*`).
TC-POST-013	EL1, EL2, LA0	**Coda vuota:** nessuna promozione e nessuna notifica di promozione (`TC-BIB58-002`, `TC-CODA-DEL-004`).
TC-POST-014	EL3, LA1, ES0, ES1	**No-show:** promozione del primo in coda; con due esecuzioni nessuna promozione né notifica duplicata (`TC-BIB47-001`, `002`, `002b`).
TC-POST-015	EL4, LA1	**Promozione scaduta:** dopo quindici minuti senza check-in la promozione scade e il posto passa al successivo (`TC-BIB47-005`).
TC-POST-016	EL0, LA1	**Slot ancora occupato:** nessuno viene promosso (`TC-CODA-VITA-020`).
TC-POST-017	SE1, RU0, OP2, ST0, MO0	**Check-in nella finestra:** accettato (prima TC-PRE-007 lo rifiutava); estremi della finestra verificati (`TC-FIN-*`); route: `TC-M2-001`.

TC ID	Test Frame (Parametri)	Oracolo / Risultato Atteso
TC-POST-018	SE1, RU0, OP4, ST1, DI0	**Estensione libera:** accettata (200) se la durata totale non supera otto ore (`TC-EST-002`, `TC-RL-EST-002`; limite: `TC-BIB27-007`); prima TC-PRE-010 la rifiutava.

Tabella 5.3: Nuovi Test Case di sistema

CAPITOLO 6

WebApp Testing (End-to-End)

Parallelamente ai System Test, è stata implementata una suite di test End-to-End (E2E) automatizzati per validare l'esperienza utente sulla WebApp. La suite è stata sviluppata con Playwright, che pilota un browser reale e simula le interazioni dell'utente (click, compilazione dei moduli, navigazione) in un ambiente controllato.

6.1 Strategia di Testing E2E

I test E2E si concentrano sui flussi delle due Change Requests, verificando l'integrazione completa tra pagine, API e database sulla build di produzione.

- **Ambiente:** build di produzione di Next.js, Chromium, lingua italiana e fuso orario di Roma; PostgreSQL 16 con migrazioni e dati dimostrativi caricati prima della suite, su un database il cui nome deve dichiarare di essere di test.
- **Esecuzione:** un solo worker, scenari in serie che condividono lo stato del database come farebbero utenti reali; tracce e screenshot conservati in caso di errore.
- **Accesso e Autorizzazione:** redirect al login e ritorno alla pagina richiesta, credenziali errate, ruoli, cookie contraffatti.
- **Happy Path:** prenotazione guidata, coda, promozione.
- **Negative Testing:** posto preso da un altro studente all'ultimo istante.
- **Data Binding:** posizione in coda, nome del promosso, notifica e prenotazione a nome del promosso.

La suite è configurata anche nella pipeline di integrazione continua, in un job dedicato, dove il job `e2e` risulta superato, ed è risultata stabile su tre esecuzioni consecutive in locale (12 scenari su 12, circa 37 secondi ciascuna).

6.2 Casi di Test E2E Automatizzati

Di seguito sono riportati i Test Case implementati nei file `tests/e2e/*.spec.ts`. Ogni test è identificato da un ID univoco E2E.

TC ID	Descrizione Scenario	Oracolo / Risultato Atteso
E2E-01	**Ritorno dopo il login.** Un anonimo apre una pagina riservata.	Redirect al login; dopo l'accesso torna alla pagina richiesta.
E2E-02	**Password errata.**	Nessuna sessione; messaggio «Email o password non corretti. Riprova.»

TC ID	Descrizione Scenario	Oracolo / Risultato Atteso
E2E-03	**Studente nell'area staff.**	La pagina amministrativa non si apre e le API dello staff rispondono 403.
E2E-04	**Cookie inventato.**	Le pagine riservate portano al login, le API rispondono 401.
E2E-05	**Prenotazione guidata.** Data, mezza giornata, sala, posto.	Conferma della prenotazione, ritrovata tra «Le mie prenotazioni».
E2E-06	**Posto occupato.** Un secondo studente apre la stessa fascia.	Il posto appare occupato e viene proposta la lista d'attesa.
E2E-07	**Ingresso in coda.**	Posizione 1, riletta dal server.
E2E-08	**Annullamento dello staff.**	L'operatore vede il nome dello studente promosso dalla coda.
E2E-09	**Notifica al promosso.**	Notifica «Posto assegnato dalla lista d'attesa» e prenotazione a suo nome.
E2E-10	**Conflitto all'ultimo istante.** Un secondo browser prende il posto durante la conferma.	Messaggio «Posto non più disponibile» con accesso alla lista d'attesa.
E2E-11	**Cancellazione del titolare.**	Chi è primo in coda ottiene il posto.
E2E-12	**Ingresso e uscita dalla coda.** Ingresso dalla mappa, uscita dalla scheda «Lista d'attesa».	Il pannello mostra la posizione 1 e la scheda il conteggio (1); dopo l'uscita compaiono «Sei uscito dalla lista d'attesa» e «Non sei in nessuna lista d'attesa», e il conteggio torna a (0).

Tabella 6.1: Casi di Test E2E